

**Автономная некоммерческая организация высшего образования
«Университет Иннополис»**

**АННОТАЦИЯ
НА ВЫПУСКНУЮ КВАЛИФИКАЦИОННУЮ РАБОТУ
(БАКАЛАВРСКУЮ РАБОТУ)
ПО НАПРАВЛЕНИЮ ПОДГОТОВКИ
09.03.01 «Информатика и вычислительная техника»**

**НАПРАВЛЕННОСТЬ (ПРОФИЛЬ) ОБРАЗОВАТЕЛЬНОЙ
ПРОГРАММЫ**

«Информатика и вычислительная техника»

Тема

**Распределенная система управления рабочими нагрузками
основанная на пулл-модели**

Выполнил

**Шамсутдинов Искандер
Рустамович**

подпись

Иннополис, 2026

Оглавление

Введение	3
Краткое описание основной части работы	7
Заключение	9
Список использованных источников	11

Введение

Актуальность темы исследования

Kubernetes [1] и HashiCorp Nomad [2] стали стандартом для запуска контейнеров в распределённой среде, однако за это приходится платить ресурсами и операционной нагрузкой на сопровождение. Даже у минимального отказоустойчивого кластера Kubernetes управляющий слой занимает порядка 13 ядер и 18,5 ГБ памяти, и под сами приложения из этих ресурсов остаётся меньше 8 %. При дальнейшем масштабировании кластера эти затраты распределяются на множество узлов и в относительном выражении становятся незначительными, однако для небольших компаний с ограниченной инфраструктурой они ощутимы, поскольку фиксированная стоимость управляющего слоя приходится на малое число рабочих узлов. Ключевую же роль играет то, что планирование остаётся за управляющим слоем: его планировщик решает, на какой узел отправить нагрузку, и лишь после этого конфигурация доставляется на узел по pull-схеме. Такая централизация делает планировщик единой точкой отказа и узким местом, а сопровождение системы обычно требует отдельной команды инженеров.

Предлагаемая альтернатива переносит планирование на узлы: управляющий слой вообще не принимает решений о размещении, а узлы сами обра-

щаются к нему, забирают свободные нагрузки и захватывают на них аренду (lease), подтверждая, что нагрузка теперь закреплена за ними [3], [4]. Координацию при этом берёт на себя реляционная база данных, а не отдельный протокол консенсуса; такой выбор сознательно отдаёт предпочтение доступности и устойчивости к разрывам сети перед строгой согласованностью [5], [6]. Оценить жизнеспособность такого подхода на практике можно только на работающем прототипе, проверенном на реальной распределённой инфраструктуре.

Цель работы

Спроектировать и реализовать прототип системы управления распределёнными нагрузками на основе pull-модели и проверить на практике, удастся ли при этом надёжно планировать нагрузки и восстанавливаться после отказов, расходуя на управляющий слой заметно меньше ресурсов, чем существующие оркестраторы.

Задачи исследования

1. Изучить Kubernetes и Nomad, оценить их эксплуатационные и ресурсные издержки и на этой основе сформулировать требования к прототипу.
2. Спроектировать децентрализованную архитектуру: управляющий слой без собственного состояния, хранение в PostgreSQL и захват нагрузок через механизм аренды.
3. Реализовать четыре компонента: управляющий слой, агент, ингресс-прокси (Edge) и консольную утилиту `mctl`.

4. Измерить на реальной инфраструктуре время планирования (SLO) и время восстановления (RTO) в пяти сценариях, по 100 прогонов в каждом.
5. Сравнить ресурсы, которые требует управляющий слой прототипа, с Kubernetes и Nomad.

Объект и предмет исследования

Объектом исследования выступает распределённая система оркестрации кратковременных нагрузок. Предметом является pull-модель координации, при которой агенты сами опрашивают управляющий слой без состояния и захватывают эксклюзивную аренду через реляционную базу данных. Работа построена по методологии Design Science Research [7], в рамках которой выявление проблемы, проектирование и оценка артефакта выполняются итеративно.

Научная новизна

1. Предложен прототип оркестратора, в котором управляющий слой не хранит состояния, осуществляет координацию через PostgreSQL, а отказоустойчивость обеспечивается одним лишь истечением аренды, без протоколов консенсуса и механизмов heartbeat.
2. Показано, что восстановление работает надёжно: средний RTO составил 47,8 с (95 % ДИ [45,8; 49,8]), все 100 прогонов завершились успешно, и система уложилась в целевые 90 с.
3. Измерена экономия ресурсов: в сопоставимой отказоустойчивой конфигу-

рации управляющему слою достаточно 1,75 ядра и 3,25 ГБ против 13 ядер и 18,5 ГБ у Kubernetes.

4. Выявлены издержки совмещения нагрузок на одном узле: два контейнера на одном узле планируются на 76 % дольше, чем разнесённые по разным узлам (31,5 с против 17,9 с).

Краткое описание основной части работы

Глава 1. Введение. Обозначает проблему, а именно высокую стоимость и уязвимость push-планирования в существующих оркестраторах, и ставит два исследовательских вопроса: можно ли с помощью pull-модели снизить издержки, не теряя в отказоустойчивости, и на какие компромиссы при этом приходится идти. Здесь же изложена методология Design Science Research.

Глава 2. Обзор литературы. Обобщает работы по оркестрации и децентрализованному планированию: от Borg [8] и Omega до распределённых Sparrow и Apollo; pull-планирование в бессерверных средах [3]; теорему CAP и архитектуры с приоритетом доступности [5], [6]; аренду как примитив координации [4]; облегчённую оркестрацию для периферийных сред [9].

Глава 3. Выявление проблемы. На примере B2B-сервиса аутентификации показано, во что обходится отказ узла: без оркестрации теряется около 3000 запросов, тогда как оркестрация сокращает RTO до 90 с. Разбираются Kubernetes и Nomad, из анализа выводятся три функциональных и четыре нефункциональных требования, среди них RTO не более 90 с и mTLS на всех каналах между компонентами.

Глава 4. Реализация. Объясняет, почему выбрана pull-модель, и опи-

сывает деление системы на управляющий слой и слой данных. Управляющий слой не хранит состояния, опирается на PostgreSQL и взаимодействует по REST поверх HTTPS с mTLS. Шесть технических сценариев иллюстрируют полный цикл работы с нагрузкой: создание, опрос агентом, захват аренды, жизненный цикл контейнера, истечение аренды и маршрутизацию через Edge. Исходный код опубликован в открытом доступе.

Глава 5. Оценка. Описывает стенд в Selectel (3 узла по 2 ядра и 4 ГБ), пять сценариев по 100 прогонов и метрики SLO и RTO. Приводит результаты, критерий Манна–Уитни ($p < 0,001$), сравнение по ресурсам и пять угроз достоверности.

Глава 6. Заключение. Подводит итоги по обоим исследовательским вопросам, называет сильные стороны решения, его ограничения и дальнейшие направления.

Заключение

Прототип на pull-модели планирует нагрузки предсказуемо: одиночную нагрузку он запускает в среднем за 14,2 с (95 % ДИ [13,8; 14,6]), пару на разных узлах за 17,9 с, две нагрузки на одном узле за 31,5 с. После отказа узла система восстанавливается за 47,8 с в среднем (95 % ДИ [45,8; 49,8]; P95 = 59,3 с) и во всех 100 прогонах укладывается в целевые 90 с. Критерий Манна – Уитни подтверждает, что накладные расходы значимы и при распределённом, и при одноузловом планировании ($p < 0,001$). За 700 размещений нагрузок не случилось ни одного конфликта аренды и ни одного двойного захвата.

Управляющему слою достаточно 0,5 ядра и 1 ГБ памяти, а в отказоустойчивой конфигурации требуется 1,75 ядра и 3,25 ГБ; это в разы меньше Kubernetes (13 ядер, 18,5 ГБ) и ниже Nomad. Экономия достигается тем, что управляющий слой не хранит состояния: его реплики взаимозаменяемы, им не нужны ни консенсус, ни выбор лидера, и предел масштабирования упирается в пропускную способность PostgreSQL, а не в саму модель оркестрации.

Простота решения сопряжена с рядом ограничений. Планировщик работает по принципу «первым пришёл, первым обслужен», а 30-секундный таймаут аренды задаёт нижнюю границу скорости обнаружения отказа (значение настраивается). Экспериментальная проверка охватила лишь два агента и две

нагрузки на узел, минимальные образы контейнеров и один сетевой регион. В дальнейшем целесообразно проверить систему на десятках агентов и сотнях нагрузок, выполнить длительные прогоны для поиска утечек и добавить правила выбора узлов, поэтапные обновления (rolling updates), абстракцию реплик и телеметрию.

Список использованных источников

- [1] Kubernetes Authors, *Kubernetes Documentation: Architecture Overview*, <https://kubernetes.io/docs/concepts/architecture/>, Accessed: 31 October 2025, 2025.
- [2] HashiCorp, *Nomad Documentation: Architecture Overview*, <https://developer.hashicorp.com/nomad/docs/architecture>, Accessed: 31 October 2025, 2025.
- [3] S. Akbari, M. Hauswirth и др., “Hiku: Pull-Based Scheduling for Serverless Computing,” в *Proceedings of the IEEE 25th International Symposium on Cluster, Cloud and Internet Computing*, IEEE, 2025. DOI: 10.1109/CCGRID64434.2025.00034. url: %5Curl%7Bhttps://arxiv.org/abs/2502.15534%7D.
- [4] C. G. Gray и D. R. Cheriton, “Leases: an efficient fault-tolerant mechanism for distributed file cache consistency,” в *Proceedings of the 12th ACM Symposium on Operating Systems Principles*, ACM, 1989, с. 202—210. DOI: 10.1145/74850.74870.
- [5] E. Brewer, “CAP Twelve Years Later: How the Rules Have Changed,” *IEEE Computer*, т. 45, № 2, с. 23—29, 2012. DOI: 10.1109/MC.2012.37.

- [6] G. D. et al., “Dynamo: amazon’s highly available key-value store,” в *Proceedings of the 21st ACM Symposium on Operating Systems Principles*, ACM, 2007, c. 205—220. DOI: 10.1145/1323293.1294281.
- [7] A. Dresch, “Design Science Research,” 2014, [Online]. Available: https://link.springer.com/chapter/10.1007/978-3-319-07374-3_4.
- [8] A. V. et al., “Large-scale cluster management at Google with Borg,” в *Proceedings of the Tenth European Conference on Computer Systems*, ACM, 2015, c. 1—17. DOI: 10.1145/2741948.2741964.
- [9] I. C. et al., “Performance Evaluation of Container Orchestration Tools in Edge Computing Environments,” *Sensors*, т. 23, № 8, c. 4008, 2023. DOI: 10.3390/s23084008.